

ACCESO A DATOS

Proyecto integrador - Cuaderno de notas

Funcionalidad CRUD e intercambio de datos entre diferentes tipos de almacenamiento

Cada pareja o grupo de alumnos elige una entidad sobre la que almacenar información (actores, personajes de un videojuego, vehículos, mascotas, etc.). Vamos a implementar una aplicación que irá creciendo a lo largo del curso de manera que vayamos sumando funcionalidad CRUD (Create, Read, Update, Delete) para cada uno de los tipos de acceso a datos que están incluidos en el temario (ficheros, bases de datos, uso de herramientas objeto-relacionales, etc.)

FUNCIONALIDAD V1.1 – Versión básica: ficheros, ficheros binarios y ficheros XML.

1. Funcionalidad CRUD
 - 1.1. Mostrar los datos del documento (todos)
 - 1.2. Añadir uno
 - 1.3. Modificar uno
 - 1.4. Borrar uno
 - 1.5. Buscar uno (por id)
2. Paso de información de un acceso a datos a los demás (“copias de seguridad”)

En los primeros días hemos estado repasando ficheros de texto, probando y haciendo alguno de los puntos dejando para más adelante la funcionalidad completa de esta primera entrega. Antes de continuar....

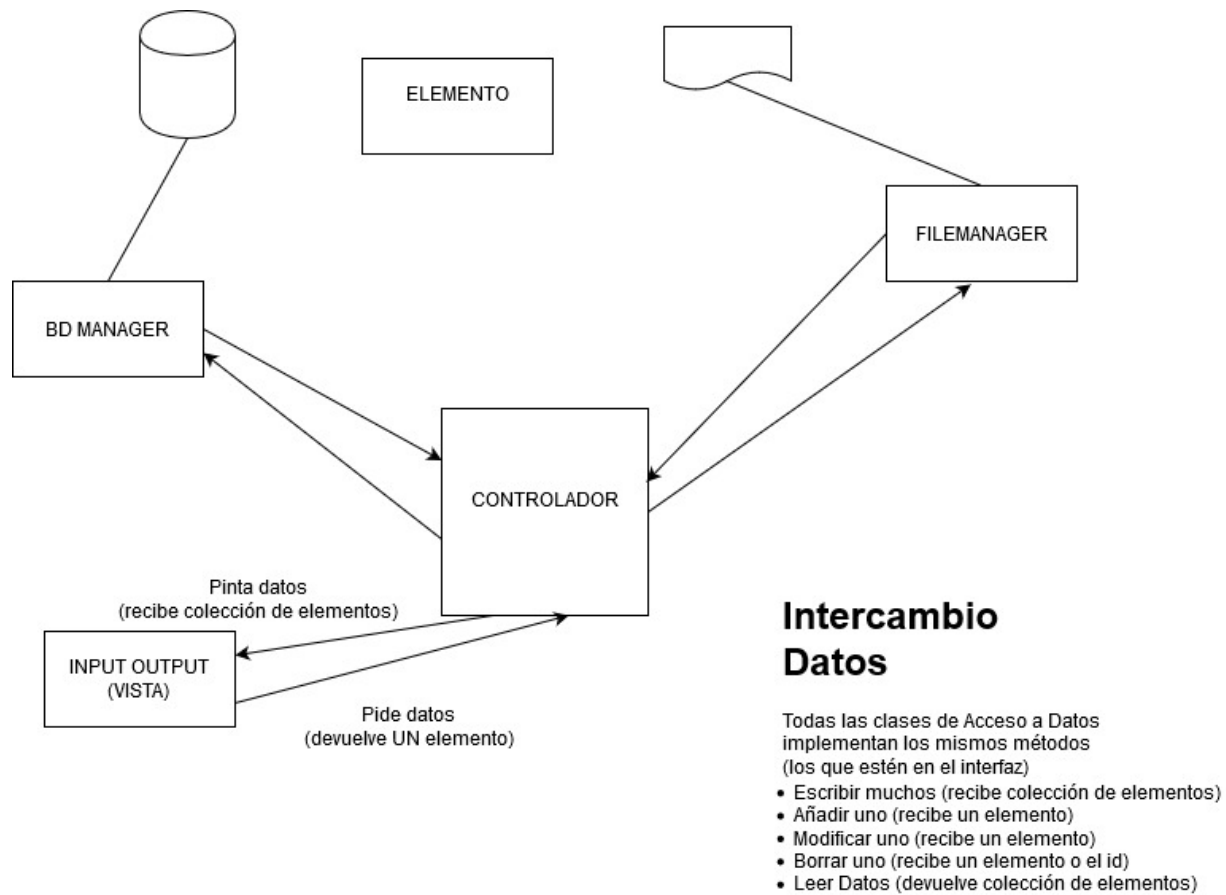
"PARAMOS" Y MÁQUINAS Y REFLEXIONAMOS SOBRE LO QUE ESTAMOS HACIENDO (16/09/2024)

- Enunciado: funcionalidad CRUD (Create, Read, Update, Delete) e intercambio de información entre diferentes tipos de acceso a datos (ahora mismo XML y ficheros, pero ya sabemos que van a ser más en el futuro).
- Por ahora estamos tratando con una "entidad" (vuelos, alumnos, la que cada uno haya escogido) pero podrían ser más en el futuro (se pueden incluir otras entidades relacionadas).
- ¿Estamos organizando bien nuestro código? ¿estamos escribiendo nuestro código pensando en que sea reutilizable?
 - ¿Es fácil añadir un nuevo tipo de acceso a datos?
 - ¿Es fácil añadir una nueva entidad?
 - ¿Hay que realizar muchos cambios?
 - ¿Hay que reescribir (copiar y pegar) mucho código?

Algunas claves:

- Fichero(s) de configuración (.ini) para rutas y otras propiedades de acceso. Clase que se encargue de manejar ese tipo de ficheros (recomendación: usar PROPERTIES)
- Clase para cada entidad representada (por ahora una, puede que haya más en el futuro) con sus atributos y sus getters y setters. Llamamos entidad a actores, personajes de un videojuego, vehículos, mascotas, etc. Basta con que tenga 3 o 4 atributos (identificador obligatorio)
- Clase encargada para cada tipo de acceso a datos. Cada clase está especializada en un acceso a datos y **no sabe nada del resto**
 - **¿Se hace en todas las clases lo mismo? => Definir interface**
 - Todas las clases que sean de acceso a datos (ahora mismo ficheros y xml) implementarán el interfaz
- Dos menús
 - 1 para elegir con quien trabajamos (que acceso a datos vamos a utilizar)
 - 2 para elegir qué es lo que queremos hacer
- Utilización de Hashmap u otra estructura de datos que nos facilite el movimiento de información entre clases (recomendamos hashmap porque no es necesario recorrerla para encontrar un elemento)
- Separamos la vista (incluso la consola) del resto => implementación de un MVC
- Prohibido acceder a los atributos de un objeto sin utilizar un método
- No se recomienda el uso de métodos estáticos
- Buena utilización de POO => constructores para los objetos, uso de parámetros, etc.
- Palabras clave: reutilización (una tarea sólo la hago una vez), abstracción (uso de interfaces para "olvidarme" de quien lo hace), divide y vencerás (una tarea grande la divido en más pequeñas: al principio trabajo más pero mi programa estará mejor estructurado y cada una de las partes será más fácil de modificar o sustituir en el futuro)

ESQUEMA GENERAL



Ejemplo de “divide y vencerás” y reutilización:

Si en BD Manager tenemos un método leerTodos que nos devuelve un hashmap de objetos de la clase vuelo, podemos desde el controlador llamar a ese método tanto si es para escribir los datos en la vista como para pasar los datos al fichero a través del FileManager

Si en la vista (ya sea gráfica o por consola) tenemos un método que recorre un hashmap de vuelos y los muestra, podemos utilizar ese método tanto para mostrar los datos que haya en la BD como en el fichero (si ampliamos y añadimos un nuevo acceso a datos también nos sirve)

PROPUESTA DE MENU (se puede hacer de mil formas, esta solo es una de ellas ... eso sí, le saca partido al uso de interfaces porque una vez elegido el acceso a datos, se reutiliza todo el código del controlador)

MENU 1

¿Qué acceso a datos primario quieres utilizar?

- 1- Fichero
- 2- XML

MENU 2

¿Qué quieres hacer?

- 1- Leer
- 2- Añadir uno
- 3- ...
- 5- Importar datos a...
 - a. ¿A qué acceso a datos quieres pasar los datos?
 - i. Ficheros
 - ii. XML
- 6- Exportar datos de...
 - a. ¿De qué acceso a datos quieres pasar los datos?
 - i. Ficheros
 - ii. XML

- Nota => en las opciones 5 y 6 se podría filtrar de alguna forma y que no aparezca el acceso a datos que se ha seleccionado como primario (si ya se ha elegido Fichero que solo aparezca XML y viceversa). Basta con poner un if

EJEMPLO DE USO DE INTERFAZ EN JAVA (PARA EL DOBLE MENÚ)

```
public class Controlador {

    public void ejecucion() {

        // El objeto miAcceso tendrá implementados los
        // métodos del interfaz independientemente de la clase a la
        // que pertenezca

        Funcionalidad miAcceso;

        int opcion;

        // MENU 1 - Con quien quiero trabajar
        // Leo la opcion que me hayan escrito por teclado

        switch(opcion) {
            case 1:
                miAcceso = new BDManager();
                break;
            case 2:
                miAcceso = new FileManager();
                break;
            case 3:
                miAcceso = new MongoManager();
                break;
        }

        // MENU 2
        // Que es lo que quiero hacer con el AD seleccionado antes
        // Cualquiera que sea la clase va a tener los métodos

        switch(opcion) {
            case 1:
                //Pedir datos a la vista para crear el objeto vuelo
                vuelo = pedirDatosNuevoVuelo();
                miAcceso.insertar(vuelo);
                break;
            case 2:
                // Pedir el id que se quiere borrar
                id = pedirID();
                miAcceso.borrar(id);
                break;

                // Habría un case por cada opción del menú
        }
    }
}
```

```

public interface Funcionalidad {

    public void insertar(Vuelo vuelo);
    public void borrar (int id);
    // Aquí vendrían más métodos

}

public class FileManager implements Funcionalidad {

    @Override
    public void insertar(Vuelo vuelo) {
        // Abrir el fichero
        // Crear un nuevo string
        // Escribir ese string al final del fichero

    }

    @Override
    public void borrar(int id) {
        // TODO Auto-generated method stub

    }

}

```

Todas las clases de acceso a datos (XML, BD, Mongo, la que sea) implementan los mismos métodos (tienen la misma funcionalidad)

OTRO EJEMPLO DE USO DE INTERFAZ EN JAVA (PARA EL PASO DE INFORMACIÓN DE UN ACCESO A DATOS A OTRO)

- Nota => Por entidad se entiende la clase que representa al Vuelo
- Nota => Se recomienda como estructura de datos el hashmap pero sería parecido con listas, arrays, etc.

```

public interface AccesoDatos() {

    // Recorre la lista que le pasan por parámetro y almacena los datos
    public void escribeTodos(HashMap<tipoclave,Entidad> lista);

    // Lee datos y crea una lista
    public HashMap<clave, entidad> leeTodos();

    /* En la interface iremos añadiendo el resto de los métodos
    para completar la funcionalidad CRUD */

}

```

```

public class BDManager() implements AccesoDatos{
    public escribeTodos(HashMap<tipoclave, Entidad> lista) {
        // Código concreto para almacenar datos en la BBDD
        // Recorre el hashmap y por cada elemento que haya, genera la
        //query que corresponda y la ejecuta
    }
    public HashMap<tipoclave, Entidad> leeTodos(){
        HashMap<entidad> auxiliar;
        // Código concreto para leer datos de una BBDD y generar el
        //hashmap
        return auxiliar
    }
}

```

// La clase FileManager tendría la misma estructura

```

public class Controlador () {

    ....

    ...

    public intercambia_datos(int opcion){

        HashMap<tipoclave, Entidad> lista;

        /* Se podría hacer new de cualquier clase que implemente la
        interface */

        if(opcion == 1){

            AccesoDatos emisor = new FileManager();

            AccesoDatos receptor = new BDManager();

        else{

            AccesoDatos emisor = new BDManager();

            AccesoDatos receptor = new FileManager();

        }

        lista = emisor.leeTodos();

        Receptor.escribeTodos(lista);

    }

}

```

- Nota => Si se utiliza el menú sugerido anteriormente el emisor estaría creado y solo habría que crear un objeto para el receptor

Comentario final => este documento es una guía con diferentes ideas para hacer lo que se considera un mejor código (más organizado, más fácil de reutilizar y modificar, etc.), pero no es un ejemplo completo ni concreto de código. En otras palabras, no se puede copiar y pegar y modificar 3 o 4 cosas para que funcione. Corresponde al alumno, coger las diferentes ideas y “hacer el puzle” (preguntando siempre al profesor cuando haya un atasco)

Como se dijo en clase, no es obligatorio seguir todas las pautas ya que por encima de todo debe primar la funcionalidad.

Por ejemplo: no sé manejar bien los hashmap => puedo usar arrays u otra estructura. No entiendo bien la ventaja de usar interfaces: lo puedo dejar para una futura versión.